PARKING LOT ENTRY and EXIT CONTROL

Team 12

Marko Disic 110170552

Rahul Bhattacharjee 110167627

Abderrahmene Naceri 110174389

Hussein Nassar 110163952

Adam Zaitoun 110166501

Harkirat Maan 110178653

Workload distribution:

Program 1: Abderrahmene, Hussein, and Harkirat.

Program 2: Marko, Rahul and Adam

Program 3: All members of the group

Introduction

Managing congestion at public parking facilities is a significant challenge for municipalities, especially when changes to staffing or access control policies must be evaluated. Conducting real-world experiments can be costly, disruptive, and impractical. To address this issue, this project develops an object-oriented software simulation system that models vehicle arrivals, gate staffing configurations, and overall parking lot flow. The simulator allows different policies to be tested in a controlled environment, enabling data-driven decisions to improve efficiency and reduce congestion.

Problem Statement

The municipality needs a way to evaluate whether revised access control and staffing policies reduce congestion at a public parking lot's entry and exit points. Because real-world testing would be costly and disruptive, the goal is to develop an object-oriented software simulator that models vehicle flow under different staffing configurations.

Given: Fixed number of parking spaces, Two entry gates, and two exit gates, Not all gates are always staffed

Time Spent Depends On:

- Whether the driver has a prepaid permit or requires a ticket
- The method of payment (cash, debit card, or permit validation)
- The number of vehicles currently waiting
- A small random factor

Given Assumptions:

- Ticket issuance takes 25 seconds, Cash payment takes 90 seconds, Debit card payment takes 45 seconds
- Permit validation takes 15 seconds. Vehicles arriving when the lot is full may wait for a space to become available or leave immediately

Program 1: Vehicle Arrival Generator – Abderrahmene and Hussein

Purpose: Creates a file listing vehicles in order of arrival time, intended parking duration, and payment method. The Vehicle Arrival Generator creates a structured input file listing vehicles in order of:

- Arrival time, Intended parking duration, Payment method (Cash, Debit, Permit Validation), Permit status (Has Permit or Needs Ticket)

This file will be used by the Simulation Program to evaluate congestion under different staffing levels.
Main Classes and Responsibilities:

1. Vehicle: Responsibility: Represents a single vehicle arrival in the system.

Attributes: vehicleID, arrivalTime (in seconds), parkingDuration (in seconds), paymentMethod (Cash, Debit, PermitValidation), hasPermit (true/false)
Methods: getArrivalTime(), getParkingDuration(), getPaymentMethod(), isPermitHolder(), toString() (formats vehicle data for file output)

2. ArrivalPattern: Responsibility: Defines how vehicles arrive and how their attributes are generated.

Attributes: simulationDuration, arrivalRate (vehicles per hour), permitProbability, minParkingDuration, maxParkingDuration, randomSeed, random number generator (mt19937)
Methods: generateInterArrivalTime(), generateParkingDuration(), generatePermitStatus(), generatePaymentMethod(). This class ensures all randomness is controlled and reproducible.

3. VehicleGenerator: Responsibility: Generates and stores all vehicle objects.

Attributes: vector of Vehicle objects, ArrivalPattern object
Methods: generateVehicles(), sortByArrivalTime(), exportToFile(filename), clearVehicles()

4. FileManager Responsibility: Handles writing vehicle data to a file.

Methods: writeVehiclesToFile(vector<Vehicle>, filename),

5. GeneratorApp (Controller): Responsibility: Handles user interaction and coordinates the process.

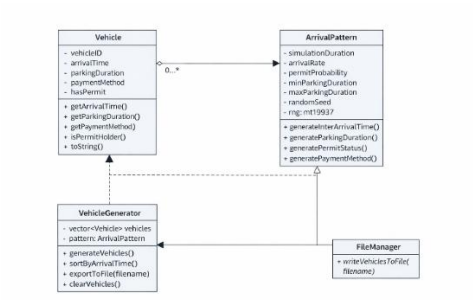
Methods: configureGeneration(), createVehicleList(), saveFile(), loadConfiguration()

<u>System Flow</u>:

START
↓User selects "Generate Vehicle Arrival File"
↓User inputs: Simulation duration, Average arrival rate, Permit probability, Parking duration range,

- Random seed, ↓System initializes ArrivalPattern using seed
  ↓VehicleGenerator generates vehicles
  ↓Vehicles are sorted by arrival time
  ↓System exports vehicle list to file
  ↓END

UML Class diagram:



<u>Functional Requirements</u>

The system must:

- Allow user to define simulation duration, Allow user to define average vehicle arrival rate, Allow user to define probability of permit holders, Allow user to define parking duration range, Allow user to define a random seed, Generate vehicles in chronological order,
- Assign each vehicle: Arrival time, Parking duration, Permit status, Payment method
- Use pseudo-random number generation, Produce identical results when using the same seed and inputs, Save output to a structured file readable by the simulation program

<u>Non-Functional Requirements</u>

Performance: Vehicle generation must run in linear time relative to number of vehicles, File writing must complete efficiently.

Usability: Clear user prompts, Simple and readable file format, Informative error messages for invalid inputs.

Reliability: No duplicate vehicle IDs, Arrival times must be sorted, Same seed produces identical output.

Maintainability: Clear separation between data generation and file handling, Modular object-oriented design, Easy to modify arrival distributions later.

Algorithms: Algorithm 1: Generate Vehicle Arrivals

1. Set currentTime = 0.
2. While currentTime < simulationDuration:
    - Generate random inter-arrival time, Add to currentTime, If currentTime exceeds simulation duration, stop. Generate vehicle attributes. Add vehicle to list.
3. End loop.

Algorithm 2: Generate Parking Duration

1. Generate random value between minimum and maximum duration.
2. Assign value to vehicle.

Algorithm 3: Assign Permit and Payment Method

1. Generate random number between 0 and 1.
    - If value < permitProbability: hasPermit = true, paymentMethod = PermitValidation
    - Else: hasPermit = false, Randomly assign Cash or Debit.

Algorithm 4: Export to File

1. Open output file.
2. Write header information: Simulation duration, Random seed, Total vehicles
3. For each vehicle: Write vehicleID, arrivalTime, parkingDuration, permitStatus, paymentMethod.
4. Close file.

Libraries and Tools (C++)

- iostream for user input, fstream for file input/output, vector for storing vehicles, string for text handling, random for pseudo-random number generation, algorithm for sorting

Program 2: Gate Staffing Configuration Generator - Marko and Rahul

Purpose: Specifies when entry and exit gates are staffed and unstaffed.

Main Classes and Responsibilities:

1. Gate
   Responsibility: Represents a physical gate in the parking lot.
   Attributes: gateID, gateType (Entry or Exit), currentStatus.
   Methods: setStatus(), getStatus().

2. TimeInterval
   Responsibility: Represents a time period during which a gate has a specific staffing status.
   Attributes: startTime, endTime, status (Staffed or Unstaffed).
   Methods: isWithinInterval(time), getStatus().

3. StaffingSchedule
   Responsibility: Stores and manages all staffing intervals for all gates.
   Attributes: List of Gate objects and a mapping of Gate to List of TimeInterval.
   Methods: addInterval(gate, interval), getGateStatus(gate, time), validateSchedule(), exportToFile().

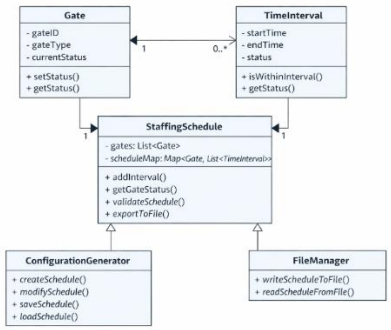
4. ConfigurationGenerator
   Responsibility: Handles user interaction and builds staffing schedules.
   Methods: createSchedule(), modifySchedule(), saveSchedule(), loadSchedule().

5. FileManager
   Responsibility: Manages file input and output.
   Methods: writeScheduleToFile(), readScheduleFromFile().

<u>System Flow</u>:

```
START
↓User selects "Create New Schedule"
↓System initializes empty StaffingSchedule
↓For each gate:
    ↓User inputs time intervals
    ↓System creates TimeInterval objects
    ↓System validates intervals for overlap
    ↓
Are there more gates?
    → YES: Repeat gate process
    → NO: Continue
↓System validates full schedule
↓Is schedule valid?
    → NO: Return to gate input to fix errors
    → YES: Continue
↓Schedule is saved to file
↓END
```

<u>UML Class Diagram</u>



Functional Requirements:

The system must allow the user to define staffing schedules for Entry Gate 1, Entry Gate 2, Exit Gate 1, and Exit Gate 2. It must allow the user to define start and end times and specify staffed or unstaffed status. The system must prevent overlapping time intervals for the same gate, validate that the start time is earlier than the end time, save the configuration to a structured file, load existing configurations for editing or reuse, and support multiple different staffing policies.

Non-Functional Requirements:

Performance: Schedule validation must execute efficiently (linear with number of intervals) and file operations must complete without noticeable delay.

Usability: The system must provide clear prompts, a readable file format, and informative error messages for invalid inputs.

Reliability: The system must prevent invalid or overlapping intervals and ensure no data corruption during file writing.

Maintainability: The design must be modular and object-oriented with clear separation between data storage and file handling.

Algorithms:

Algorithm 1: Add Staffing Interval

1. Receive gate ID, start time, end time, and status.
2. Verify start time is earlier than end time.
3. Retrieve existing intervals for the gate.
4. Check for overlap; if the new interval overlaps any existing interval, reject it.
5. If valid, add the interval to the gate's list.
6. Sort intervals by start time.

Algorithm 2: Get Gate Status at Specific Time

1. Retrieve the interval list for the gate.
2. For each interval, if currentTime is between startTime and endTime, return the interval status.
3. If no interval matches, return the default status (Unstaffed).

Algorithm 3: Validate Entire Schedule

1. For each gate, ensure intervals do not overlap and ensure all times are within operating hours.
2. If all checks pass, return valid.
3. Otherwise, return error.

Libraries and Tools:

Libraries: <iostream> for user interaction, <fstream> for file input/output, <vector> for dynamic storage of intervals, <map> to associate gates with intervals, and <string> for time and status handling.

Program 3: Simulation Program Adam and

Purpose: Reads both files and simulates vehicle flow through the parking lot. The simulation evaluates congestion and system performance under different staffing policies.

Requirements: Average wait time at entry and exit gates, Gate utilization and idle time, Number of vehicles turned away, Parking space utilization over time

Main Classes and Responsibilities:

1. Simulation: Responsibility: Controls the overall simulation process.

Attributes: currentTime, simulationEndTime, ParkingLot parkingLot, vector<Gate> entryGates, vector<Gate> exitGates, priority_queue<Event> eventQueue, StatisticsCollector statistics

Methods: loadArrivalFile(filename), loadStaffingFile(filename),initializeSimulation(), run(), processNextEvent(), reportResults()

2. ParkingLot: Responsibility: Manages parking space capacity and vehicle occupancy.

Attributes: totalSpaces, availableSpaces, list of parked vehicles

Methods: isFull(), parkVehicle(vehicle), removeVehicle(vehicle), getUtilization(), getAvailableSpaces()

3. Gate: Responsibility: Represents an entry or exit gate during simulation.

Attributes: gateID, gateType (Entry or Exit), isStaffed, queue<Vehicle>, busyUntilTime, totalBusyTime, totalIdleTime

Methods: isAvailable(currentTime), addVehicleToQueue(vehicle), processVehicle(currentTime), updateStatus(time), getUtilization()

4. Event: Responsibility: Represents simulation events in chronological order.

Attributes: eventTime, eventType (Arrival, EntryComplete, ExitComplete, GateStatusChange), vehicleID, gateID

Methods: getEventTime(), getEventType(). This class supports event-driven simulation using a priority queue.

5. StatisticsCollector: Responsibility: Tracks and computes performance metrics.

Attributes: totalEntryWaitTime, totalExitWaitTime, vehiclesProcessed, vehiclesTurnedAway, cumulativeParkingUtilization

Methods: recordEntryWait(time), recordExitWait(time), incrementTurnedAway(), calculateAverages(), generateReport()

6. FileManager: Responsibility: Reads input files from Program 1 and Program 2.

Methods: readVehicleFile(filename), readStaffingFile(filename)

System Flow:

```
START
↓Load vehicle arrival file
↓Load staffing configuration file
↓Initialize parking lot and gates
↓Insert all vehicle arrival events into event queue
↓Set currentTime = 0
↓While eventQueue not empty:
    Process next event (smallest eventTime)
    Update system state
    Update statistics
↓End simulation
↓Generate performance report
↓END
```

Functional Requirements:

The system must:

- Read vehicle arrival data from file, Read gate staffing configuration from file, Simulate vehicle entry and exit events chronologically, Model gate queues, Prevent vehicles from entering if parking lot is full, Allow vehicles to wait or leave if full, Handle gate closures while vehicles are queued.
- Track service time based on: Ticket issuance (25 seconds), Cash payment (90 seconds), Debit payment (45 seconds), Permit validation (15 seconds)
- Include a small random processing factor.
- Calculate: Average entry wait time, Average exit wait time, Gate utilization, Idle time, Number of vehicles turned away, Parking utilization over time

Non-Functional Requirements:

Performance: Event processing must be efficient (priority queue), Simulation runtime must scale with number of events.

Usability: Clear output report, Structured readable summary, Error handling for invalid input files.

Reliability: No negative parking capacity, Events processed strictly in chronological order, Reproducible results with same inputs.

Maintainability: Clear separation of: Event handling, Gate logic, Parking lot logic, Statistics collection, Modular object-oriented design.

Algorithms:

Algorithm 1: Event-Driven Simulation Loop

1. Insert all arrival events into priority queue.
2. While queue not empty:
    - Remove earliest event, Set currentTime = eventTime,
    - Process event based on type:
        - Arrival, EntryComplete, ExitComplete,GateStatusChange
3. Continue until queue empty.

<u>Algorithm 2: Vehicle Arrival Processing</u>

1. Check if parking lot is full.
2. If full:
    - Vehicle waits or leaves (based on rule), Update turnedAway count if leaving.
3. If not full:
    - Choose best staffed entry gate:
        - Select gate with smallest queue, If tie, randomly select.
    - Add vehicle to gate queue, If gate free, schedule EntryComplete event.

<u>Algorithm 3: Entry Completion:</u>

1. Park vehicle. Schedule Exit event at: currentTime + parkingDuration
2. If queue not empty: Start next vehicle service.
3. Update gate busy time.

<u>Algorithm 4: Exit Completion:</u>

1. Remove vehicle from parking lot, Update availableSpaces, If exit queue not empty:
    - Process next vehicle.
2. Update statistics.

<u>Algorithm 5: Gate Status Change</u>

1. Update gate staffing status.
    - If gate closes: Stop accepting new vehicles, Continue serving queued vehicles.
    - If gate opens: Accept vehicles again.

<u>Gate Selection Logic:</u> Vehicles choose:

1. All staffed gates of required type. Select gate with minimum: queueLength × averageServiceTime.
    - If tie: Randomly select one.

<u>Libraries and Tools</u>: Iostream, fstream, vector, queue, priority_queue, string, random, algorithm, map

Questions:

1. How does a vehicle choose which gate to enter or exit?
   A vehicle can choose entry or exit gate by looking at all the staffed and open gates that follow whether or not they want to enter or exit. They can then select the ones that have the lowest expected queue, we can calculate that from the number of cars in queue and how long it takes to serve them (ticket issuance;25 seconds, cash payments; 90 seconds, debit; 45s, permit 15s.)
   If we have a tie we can just have the code randomly select a winner as a resolution.
2. What happens if a gate closes while vehicles are queued?
   If a gate were to close on cars that are queued, we can ensure that the gate stops accepting new cars and get those that were already queued taken care of until the queue is empty.
3. Should permit holders be prioritized?
   No, permit holders should not be automatically prioritized. Instead, we can have permit prioritization be determined by a policy that looks at priority or dedicated gates that can operate without staff. Since we are writing this code for a municipality, we want to keep it as open as possible so that we can experiment with the simulation so we can evaluate how permit priorities affect efficiency.
4. How long does it take to reassign staff between gates?
   Reassigning staff between gates requires a transition period to account for walking time and setup. During this time, the staff member is unavailable and cannot operate either gate. A fixed transition time (five minutes) will be used in the system. When validating schedules, the program will ensure that no staff member is assigned to a new gate until the required transition time after their previous assignment has passed. If the transition time is not respected, the schedule will be marked as invalid.
5. How does parking duration configuration affect congestion?
   The duration configuration of our parking affects congestion by controlling how many parking spaces we can overturn to new customers. Shorter parking durations will let us get more people flowing whereas long time parking makes new spots harder to come by.

Conclusion

In conclusion, this project provides a structured and modular solution for evaluating parking lot congestion without requiring costly real-world testing. By separating the system into three independent but connected programs (vehicle arrival generation, gate staffing configuration, and simulation), the design ensures flexibility, maintainability, and accurate experimentation. The Vehicle Arrival Generator produces reproducible traffic patterns using controlled randomness, the Gate Staffing Configuration Generator allows multiple staffing policies to be tested and validated, and the Simulation Program integrates both inputs to measure performance metrics such as wait times, gate utilization, and vehicle turnaways. Together, these components allow the municipality to safely compare different access control and staffing strategies, analyze their impact on congestion, and make informed decisions based on data-driven results.